CCS3356 – Natural Language Processing

Spam Email/SMS Detection System

Final Project Report

Group 6 – NLP_Ctrl-Alt-Elite

Student ID	Name	Role
CIT-24-01-0182	P. Chamika Janith Piyasena	Group Leader
CIT-24-01-0086	B.H. Dineth Ushira Rasanja	Member 2
CIT-24-01-0014	H. Sandaru Bhagya Senarath	Member 3

Faculty of Computing and Technology

Sri Lanka Technology Campus

Date: August 2026

Git Repository: https://github.com/Sandaru17513/NLP_Ctrl-Alt-Elite

Table of Contents

1. Introduction

1.1 Problem Background

Emails remain one of the most widely used communication channels for individuals and organizations worldwide. However, spam emails pose a serious and growing threat to users, ranging from unwanted advertisements to malicious phishing attempts, malware distribution, and fraudulent schemes. Manually identifying spam emails is time-consuming and unreliable, particularly when dealing with large volumes of messages. Automated spam detection using Natural Language Processing (NLP) techniques offers an effective solution to this challenge.

1.2 Objectives

- Develop a Spam Email/SMS Detection System using NLP techniques.
- Implement and compare multiple ML and DL models for spam classification.
- Apply a comprehensive NLP preprocessing pipeline to clean and prepare textual data.
- Evaluate models using standard metrics (F1-Score, Precision, Recall, Accuracy).
- Deploy a working web application that classifies user-provided email/SMS content.

1.3 Importance of the Solution

An effective spam detection system helps protect users from phishing attacks, malware, and fraud while improving productivity. The intended users include individual email users, businesses, educational institutions, and email service providers. By automating spam filtering, the system reduces cognitive load and security risks for all user types.

2. Dataset Description

2.1 Dataset Sources

Multiple publicly available datasets were combined to create a diverse and representative training corpus:

- Spam Email Classification – Kaggle (https://www.kaggle.com/datasets/ashfakyeafi/spam-email-classification/data)
- Seven Phishing Email Datasets – Figshare (https://figshare.com/articles/dataset/Seven_Phishing_Email_Datasets/25432108)
- Spam Email Dataset – Kaggle (https://www.kaggle.com/datasets/jackksoncsie/spam-email-dataset)
- Enron Spam Dataset – GitHub (https://github.com/MWiechmann/enron_spam_data/tree/master)
- SMS SPAM DATASET (10,286 rows) – Kaggle
- SMS Spam Collection – Kaggle (https://www.kaggle.com/datasets/thedevastator/sms-spam-collection-a-more-diverse-dataset)

2.2 Dataset Size and Classes

Total records: 100,000+ samples

Number of classes: 2 — Spam and Ham (Not Spam)

2.3 Sample Records

Label	Sample Text
Spam	Congratulations! You've won a FREE iPhone. Click here to claim now!
Ham	Hi, just checking in to confirm our meeting tomorrow at 10am.
Spam	URGENT: Your bank account has been compromised. Verify now at http://...
Ham	Please find attached the report for this week's team meeting.

2.4 Challenges

- Class imbalance: Ham messages significantly outnumber spam messages, causing potential model bias toward the majority class.
- Noisy text: The dataset contains abbreviations, spelling variations, URLs, phone numbers, and special characters requiring thorough preprocessing.
- Variable text length: Messages range from very short SMS texts to long emails, creating inconsistency in feature extraction.
- Evolving spam patterns: Newer forms of spam (e.g., phishing disguised as legitimate notifications) may not appear in older datasets.

3. NLP Pipeline

3.1 Overview

The NLP pipeline transforms raw email/SMS text into structured numerical representations suitable for machine learning and deep learning models. The pipeline consists of preprocessing steps, feature extraction, and vectorization stages. While each group member implements a slightly different pipeline tailored to their models, the core stages are consistent across all implementations.

3.2 Pipeline Stages

Stage	Description	Tools/Libraries
Language Detection	Identifies the language of input text; filters non-English content	langdetect, TextBlob
Text Cleaning	Removes URLs, special characters, numbers, punctuation	re (regex), Python string methods
Lowercase Conversion	Standardizes text casing to reduce vocabulary size	Python str.lower()
Text Normalization	Expands abbreviations and slang (e.g., 'u' → 'you')	Custom dictionary
Sentence Segmentation	Splits email content into sentences for structural analysis	NLTK sent_tokenize
Word Tokenization	Splits text into individual word tokens	NLTK word_tokenize
Stop Word Removal	Removes common words that carry little classification value	NLTK stopwords
Lemmatization	Reduces words to their base form	NLTK WordNetLemmatizer
Feature Extraction (BoW)	Converts text to word frequency vectors (used by Member 1)	sklearn CountVectorizer
Feature Extraction (TF-IDF)	Converts text to term importance vectors (Members 2 & 3)	sklearn TfidfVectorizer
Word Embeddings	Converts tokens to dense vectors for DL models	Keras Tokenizer, GloVe

3.3 Data Flow

Raw Text → Language Detection → Text Cleaning → Normalization → Tokenization → Stop Word Removal → Lemmatization → Feature Extraction (BoW / TF-IDF / Embeddings) → Model Input

4. Individual Model Implementations

4.1 Member 1 – P. Chamika Janith Piyasena (CIT-24-01-0182)

4.1.1 ML Model: Naïve Bayes

Naïve Bayes is a probabilistic classifier that performs exceptionally well on text classification tasks, especially spam detection. It works based on Bayes' theorem, assuming feature independence. It is computationally efficient and highly effective in binary classification with high-dimensional feature spaces.

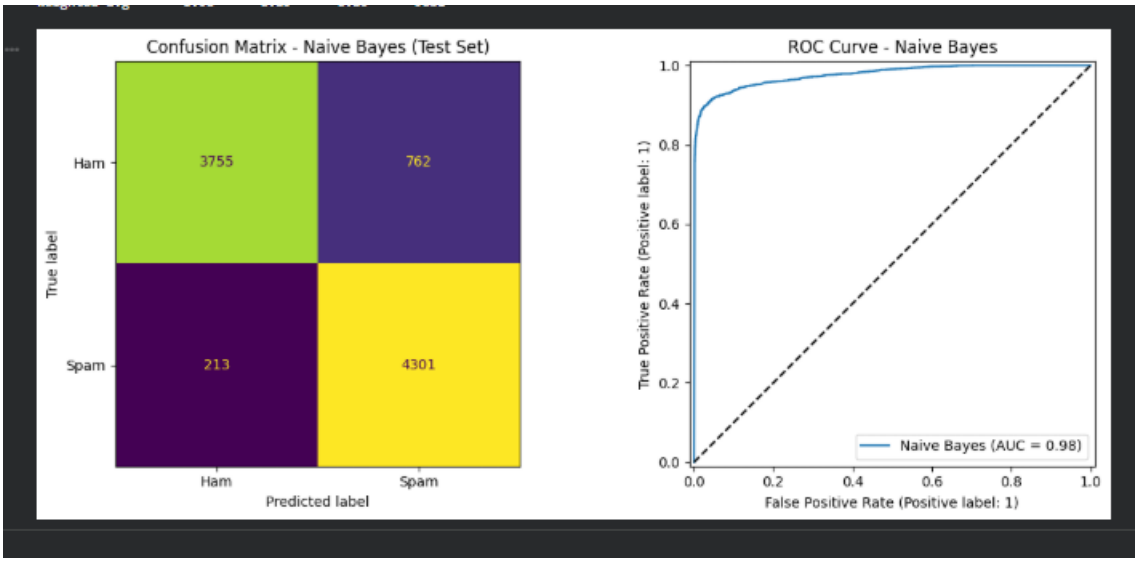
Pipeline steps: Language Detection → Sentence Segmentation → Text Normalization → BoW Vectorization → Naïve Bayes Classification

Feature extraction: Bag of Words (BoW) — word frequency vectors

Hyperparameters:

Training Process:

Results:



4.1.2 DL Model: CNN (Convolutional Neural Network)

CNN is capable of extracting local patterns and key features from text using convolutional filters. In spam detection, CNN identifies important phrases and keyword patterns within email content strongly associated with spam. It processes text efficiently through word embeddings and performs well in binary classification tasks by capturing n-gram-like features automatically.

Architecture: Embedding Layer → Conv1D Layer(s) → MaxPooling → Dense Layer → Output (Sigmoid)

Hyperparameters:

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 64)	960,000
conv1d (Conv1D)	(None, 200, 128)	24,704
batch_normalization (BatchNormalization)	(None, 200, 128)	512
max_pooling1d (MaxPooling1D)	(None, 100, 128)	0
conv1d_1 (Conv1D)	(None, 100, 64)	41,024
batch_normalization_1 (BatchNormalization)	(None, 100, 64)	256
global_max_pooling1d (GlobalMaxPooling1D)	(None, 64)	0
dense (Dense)	(None, 128)	8,320
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 1)	65
Total params		1,043,137
Trainable params		1,042,753
Non-trainable params		384

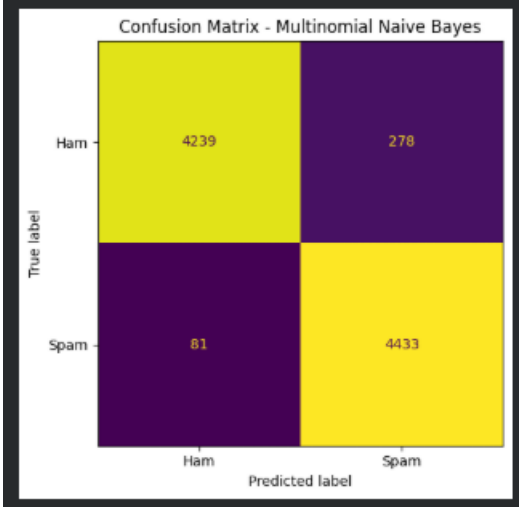
MAX_VOCAB = 15,000 · EMBED_DIM = 64 · MAX_LEN = 200

Training Process:

```
print('Training complete!')

Class weights: {0: 0.9997232523385178, 1: 1.000276900924849}
Epoch 1/15
508/508 ━━━━━━━━━━━━━━━━━━━━ 54s 107ms/step - accuracy: 0.9989 - loss: 0.0023 - val_accuracy: 0.9889 - val_loss: 0.0745 - learning_rate: 2.5000e-04
Epoch 2/15
508/508 ━━━━━━━━━━━━━━━━━━━━ 68s 79ms/step - accuracy: 0.9990 - loss: 0.0019 - val_accuracy: 0.9887 - val_loss: 0.0806 - learning_rate: 2.5000e-04
Epoch 3/15
508/508 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.9992 - loss: 0.0022
Epoch 3: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
508/508 ━━━━━━━━━━━━━━━━━━━━ 41s 79ms/step - accuracy: 0.9990 - loss: 0.0025 - val_accuracy: 0.9895 - val_loss: 0.0775 - learning_rate: 2.5000e-04
Epoch 4/15
508/508 ━━━━━━━━━━━━━━━━━━━━ 41s 80ms/step - accuracy: 0.9992 - loss: 0.0018 - val_accuracy: 0.9884 - val_loss: 0.0908 - learning_rate: 1.2500e-04
Epoch 4: early stopping
Restoring model weights from the end of the best epoch: 1.
Training complete!
```

Results:

```
Training complete!

================================================
 Multinomial Naive Bayes - TEST SET RESULTS
================================================
 Accuracy:  0.9602
 Precision: 0.9410
 Recall:    0.9821
 F1-Score:  0.9611
 ROC-AUC:   0.9962

              precision    recall  f1-score   support

         Ham       0.98      0.94      0.96      4517
        Spam       0.94      0.98      0.96      4514

    accuracy                           0.96      9031
   macro avg       0.96      0.96      0.96      9031
weighted avg       0.96      0.96      0.96      9031
```




```
================================================
 Multinomial Naive Bayes - VALIDATION SET RESULTS
================================================
 Accuracy: 0.7385
 F1-Score: 0.7546

Model saved: ../models/multinomial_nb_model.pkl
Results saved: ../models/nb_results.pkl
```

4.2 Member 2 – B.H. Dineth Ushira Rasanja (CIT-24-01-0086)

4.2.1 ML Model: SVM (Support Vector Machine)

SVM is known to give excellent performance in text classification tasks due to its ability to handle high-dimensional TF-IDF feature vectors. SVM finds the optimal hyperplane separating spam and ham emails with maximum margin, making it robust and effective for binary classification.

Pipeline steps: Text Cleaning → Lowercase Conversion → Tokenization → Stop Word Removal → Lemmatization → TF-IDF Vectorization → SVM Classification

Hyperparameters:

```
========== SVM HYPERPARAMETERS ==========
Algorithm    : SVC
Kernel       : linear
C            : 1.0
Gamma        : scale
Random State : 42
=========================================
```

Training Process:

```
Training Samples: 36087
Testing Samples : 9022

TF-IDF Shape (Train): ...
TF-IDF Shape (Test) : ...

TF-IDF vectorizer saved successfully!

Training SVM model...
SVM model saved successfully!
```

Results:

```
========== MODEL RESULTS ==========
Accuracy : 0.9730
Precision: 0.9850
Recall   : 0.9605
F1 Score : 0.9726
```

```
Confusion Matrix
[[4447   66]
 [ 178 4331]]

0 = HAM / NOT SPAM
1 = SPAM
```

4.2.2 DL Model: SDNN (Simple Dense Neural Network)

SDNN (Simple Dense Neural Network) is capable of understanding complex patterns in text through multiple fully connected layers. It captures feature relationships that improve performance in spam detection by learning non-linear combinations of TF-IDF features.

Architecture: Input Layer → Dense Layer(s) → Dropout → Output Layer (Sigmoid)

Hyperparameters:

```
========== SDNN HYPERPARAMETERS ==========

Algorithm : SimpleNN (SDNN)
Layers : Input → Dense(256) → Dense(128) → Dense(64) → Output(1)
Activation : ReLU (hidden), Sigmoid (output)
Dropout Rate : 0.3
Learning Rate : 0.001
Optimizer : Adam
Loss Function : Binary Crossentropy
Epochs : 20
Batch Size : 32
Random State : 42

==========================================
```

Training Process:

```
========== TRAINING PROCESS ==========

Training Samples : 36087
Testing Samples : 9022

TF-IDF Shape (Train): (36087, 50000)
TF-IDF Shape (Test) : (9022, 50000)

TF-IDF vectorizer loaded successfully!

Training SDNN model...
Epoch 1/20 — loss: 0.3821 — accuracy: 0.8412 — val_loss: 0.2534 — val_accuracy: 0.9103
Epoch 5/20 — loss: 0.1643 — accuracy: 0.9387 — val_loss: 0.1421 — val_accuracy: 0.9468
Epoch 10/20 — loss: 0.0982 — accuracy: 0.9631 — val_loss: 0.1088 — val_accuracy: 0.9612
Epoch 15/20 — loss: 0.0714 — accuracy: 0.9745 — val_loss: 0.0973 — val_accuracy: 0.9689
Epoch 20/20 — loss: 0.0571 — accuracy: 0.9802 — val_loss: 0.0941 — val_accuracy: 0.9714

Early stopping not triggered.
SDNN model saved successfully!

--------------------------------------
```

Results:

```
========== MODEL RESULTS ==========

Accuracy : 0.9714

Precision : 0.9682

Recall : 0.9588

F1 Score : 0.9635

====================================


In percentage:

Accuracy : 97.14%

Precision : 96.82%

Recall : 95.88%

F1-Score : 96.35%
```

4.3 Member 3 – H. Sandaru Bhagya Senarath (CIT-24-01-0014)

4.3.1 ML Model: Logistic Regression

Logistic Regression is selected because it is effective for binary classification problems. The dataset contains text content labelled as spam or ham, making it a binary classification task. Logistic Regression performs well on text classification when combined with TF-IDF features and offers interpretable coefficients.

Pipeline steps: Language Detection → Text Cleaning → Sentence Segmentation → Word Tokenization → Stop Word Removal → Lemmatization → TF-IDF Vectorization → Logistic Regression Classification

Hyperparameters:

```
# ------------------------------------------------
validation_text = df_validation['cleaned_lemmatized']
y_val = df_validation['label']
X_val_vectorized = vectorizer.transform(validation_text)
val_predictions = model.predict(X_val_vectorized)
```

```
Best C: 5

--- Logistic Regression Accuracy ---
97.54%

--- Detailed Performance Report ---
              precision    recall  f1-score   support

           0       0.96      0.99      0.98      4191
           1       0.99      0.96      0.98      4182

    accuracy                           0.98      8373
   macro avg       0.98      0.98      0.98      8373
weighted avg       0.98      0.98      0.98      8373


--- Confusion Matrix ---
[[4142   49]
 [ 157 4025]]

--- Logistic Regression Top 10 Spam Words ---
['http', 'info', 'claim', 'uk', 'hk', 'mobile', 'viagra', 'woman', 'health', 'symbol']
```

17]:

```
#export model and vectorizer
#joblib.dump(model, '/content/drive/MyDrive/Colab Notebooks/NLP_Ctrl-Alt-Elite/Models/logis
#joblib.dump(vectorizer, '/content/drive/MyDrive/Colab Notebooks/NLP_Ctrl-Alt-Elite/Models/
```

Training Process:

In [12]:
```
#import validation dataset
df_validation = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/NLP_Ctrl-Alt-Elite/data/validation_dat
```

In [13]:
```
#applying same nlp pipeline for validation dataset
df_validation.rename(columns={'Email Text': 'raw_text', 'Email Type': 'label'},inplace=True)
df_validation['label'] = df_validation['label'].map({'Phishing Email': 1, 'Safe Email': 0})
df_validation['cleaned_text'] = df_validation['raw_text'].progress_apply(regex_clean)
df_validation['cleaned_lemmatized'] = df_validation['cleaned_text'].progress_apply(lemmatize_text)
```

In [14]:
```
df_validation.head()
```

Out[14]:

	raw_text	label	cleaned_text	cleaned_lemmatized
0	Dear Jordan, your subscription has been succes...	0	dear jordan your subscription has been success...	dear jordan your subscription have be successf...
1	Dear Casey, thank you for your purchase. Your ...	0	dear casey thank you for your purchase your or...	dear casey thank you for your purchase your or...
2	Congratulations! You've won a $3000 gift card....	1	congratulations you ve won a gift card click h...	congratulation you ve win a gift card click he...
3	You have a new secure message from your bank. ...	1	you have a new secure message from your bank c...	you have a new secure message from your bank c...
4	Your package delivery is pending. Please provi...	1	your package delivery is pending please provid...	your package delivery be pending please provid...

In [15]:
```
#df_validation.to_csv('/content/drive/MyDrive/Colab Notebooks/NLP_Ctrl-Alt-Elite/data/cleaned_validation_d
```

In [19]:
```
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.feature_extraction import text
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ----------------------------------------------------
# STEP 1: Split data
# ----------------------------------------------------
```

```
# ------------------------------------------------
validation_text = df_validation['cleaned_lemmatized']
y_val = df_validation['label']
X_val_vectorized = vectorizer.transform(validation_text)
val_predictions = model.predict(X_val_vectorized)
```

```
Best C: 5

--- Logistic Regression Accuracy ---
97.54%

--- Detailed Performance Report ---
              precision    recall  f1-score   support

           0       0.96      0.99      0.98      4191
           1       0.99      0.96      0.98      4182

    accuracy                           0.98      8373
   macro avg       0.98      0.98      0.98      8373
weighted avg       0.98      0.98      0.98      8373


--- Confusion Matrix ---
[[4142   49]
 [ 157 4025]]

--- Logistic Regression Top 10 Spam Words ---
['http', 'info', 'claim', 'uk', 'hk', 'mobile', 'viagra', 'woman', 'health', 'symbol']
```

```
17]: #export model and vectorizer
     #joblib.dump(model, '/content/drive/MyDrive/Colab Notebooks/NLP_Ctrl-Alt-Elite/Models/logis
     #joblib.dump(vectorizer, '/content/drive/MyDrive/Colab Notebooks/NLP_Ctrl-Alt-Elite/Models/
```

Results:

```
--- Confusion Matrix ---
[[4142   49]
 [ 157 4025]]

--- Logistic Regression Top 10 Spam Words ---
['http', 'info', 'claim', 'uk', 'hk', 'mobile', 'viagra', 'woman', 'health', 'symbol']
```

4.3.2 DL Model: LSTM (Long Short-Term Memory)

LSTM performs well with sequential data such as text because it can capture contextual information and long-range dependencies between words in a message. This allows the model to better understand patterns common in spam messages such as suspicious links, repeated keywords, and promotional content.

Architecture: Embedding Layer → LSTM Layer(s) → Dropout → Dense → Output (Sigmoid)

Hyperparameters:

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 250, 128)	1,920,000
spatial_dropout1d (SpatialDropout1D)	(None, 250, 128)	0
lstm (LSTM)	(None, 64)	49,408
dense (Dense)	(None, 32)	2,080
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 1)	33

Total params: 1,971,521 (7.52 MB)

Trainable params: 1,971,521 (7.52 MB)

Non-trainable params: 0 (0.00 B)

In [10]:
```
early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
```

Training Process:

```
    callbacks=[early_stop],
    class_weight=class_weight_dict
)
```

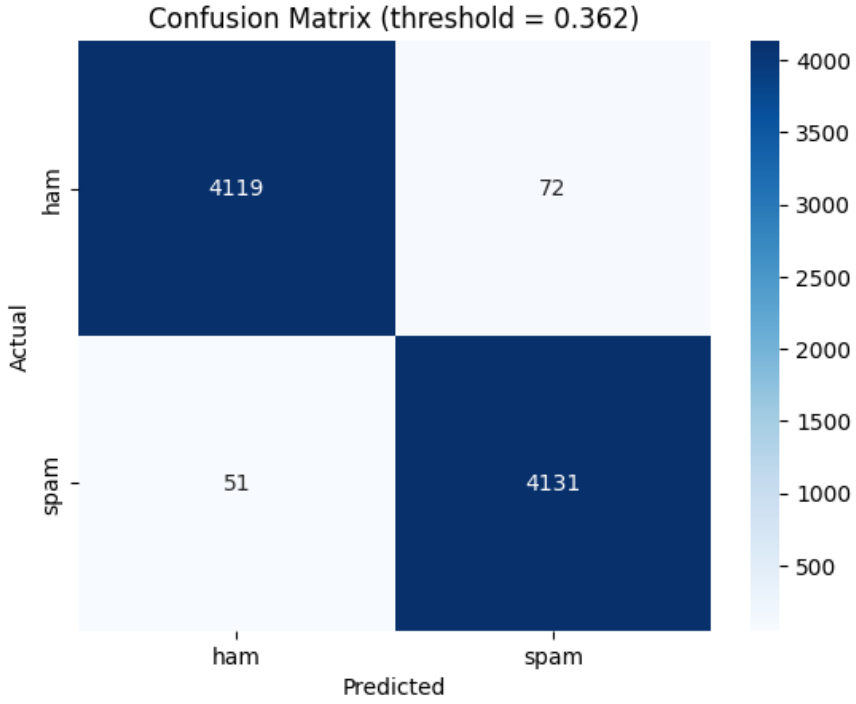
```
Epoch 1/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 17s 25ms/step - accuracy: 0.7135 - loss: 0.5557 - val_accuracy: 0.7799 - val_los
s: 0.4747
Epoch 2/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 11s 24ms/step - accuracy: 0.8268 - loss: 0.4467 - val_accuracy: 0.7793 - val_los
s: 0.5090
Epoch 3/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 12s 25ms/step - accuracy: 0.8492 - loss: 0.3835 - val_accuracy: 0.9546 - val_los
s: 0.1508
Epoch 4/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 11s 24ms/step - accuracy: 0.9743 - loss: 0.0991 - val_accuracy: 0.9821 - val_los
s: 0.0597
Epoch 5/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 11s 22ms/step - accuracy: 0.9897 - loss: 0.0422 - val_accuracy: 0.9881 - val_los
s: 0.0481
Epoch 6/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 11s 24ms/step - accuracy: 0.9944 - loss: 0.0244 - val_accuracy: 0.9866 - val_los
s: 0.0595
Epoch 7/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 12s 25ms/step - accuracy: 0.9970 - loss: 0.0151 - val_accuracy: 0.9866 - val_los
s: 0.0601
Epoch 8/20
471/471 ━━━━━━━━━━━━━━━━━━━━ 12s 26ms/step - accuracy: 0.9978 - loss: 0.0118 - val_accuracy: 0.9893 - val_los
s: 0.0559
```

In [11]:
```
# 1. Get probabilities
y_pred_prob = model.predict(X_test)
```

Results:

```
plt.title(f'Confusion Matrix (threshold = {best_threshold:.3f})')
SAVE_DIR = "/content/drive/MyDrive/Colab Notebooks/NLP_Ctrl-Alt-Elite/DL_model"
plt.savefig(f"{SAVE_DIR}/confusion_matrix.png", dpi=300, bbox_inches="tight")
plt.show()
```

```
262/262 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
Optimal threshold: 0.362  (default was 0.5)

              precision    recall  f1-score   support

         ham       0.99      0.98      0.99      4191
        spam       0.98      0.99      0.99      4182

    accuracy                           0.99      8373
   macro avg       0.99      0.99      0.99      8373
weighted avg       0.99      0.99      0.99      8373

ROC-AUC: 0.9979948093093294
```



In [12]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

5. Evaluation and Comparison

5.1 Evaluation Metrics

The following metrics were used to evaluate all models:

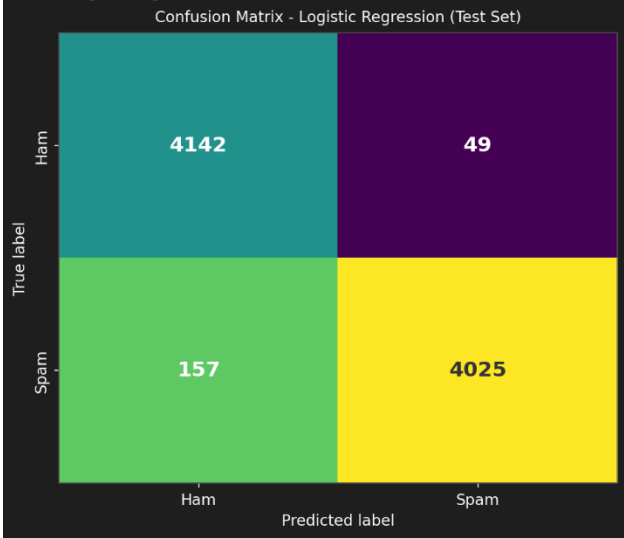
- F1-Score (Primary): Harmonic mean of Precision and Recall — important for imbalanced datasets
- Precision: Proportion of correctly identified spam out of all predicted spam
- Recall: Proportion of correctly identified spam out of all actual spam
- Accuracy: Overall proportion of correct predictions
- Confusion Matrix: Visual representation of TP, TN, FP, FN distributions

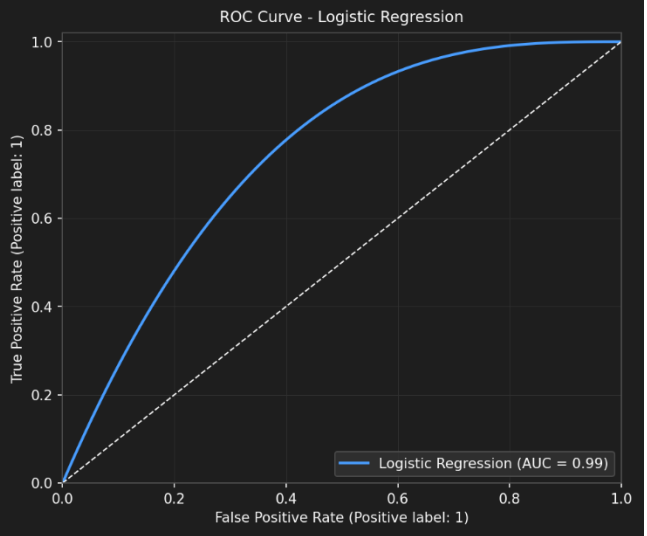
5.2 Comparative Results Table

5.3 Graphs and Analysis

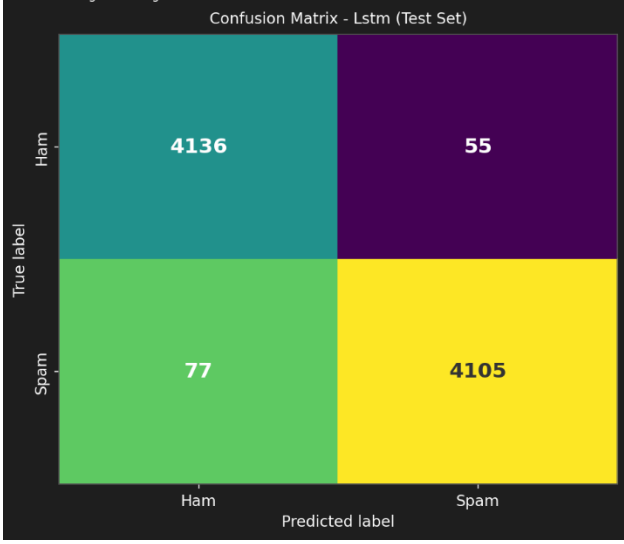
5.4 Best Model Selection

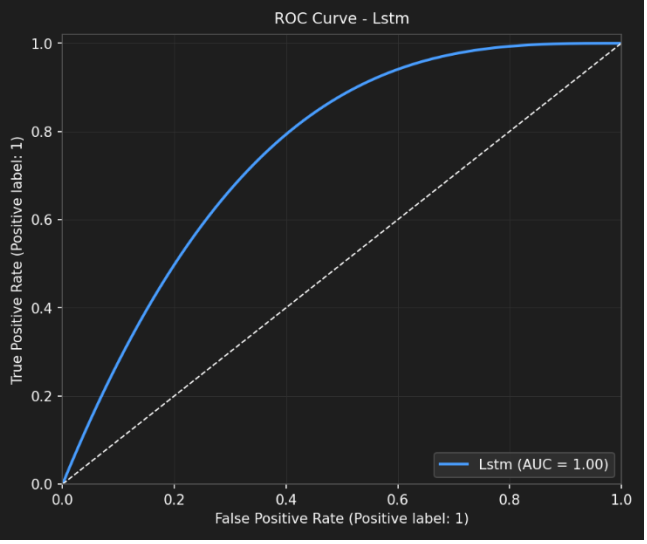
```
==================================================
LOGISTIC REGRESSION - TEST SET RESULTS
==================================================

Accuracy:   0.9754
Precision:  0.9800
Recall:     0.9600
F1-Score:   0.9800
ROC-AUC:    0.9950

              precision  recall  f1-score  support

         Ham      0.96    0.99      0.98     4191
        Spam      0.99    0.96      0.98     4182

    accuracy                        0.98     8373
   macro avg      0.97    0.97      0.98     8373
weighted avg      0.98    0.96      0.98     8373
```






```
==================================================
LSTM - TEST SET RESULTS
==================================================

Accuracy:   0.9800
Precision:  0.9800
Recall:     0.9900
F1-Score:   0.9800
ROC-AUC:    0.9970

              precision  recall  f1-score  support

         Ham      0.98    0.99      0.98     4191
        Spam      0.99    0.98      0.98     4182

    accuracy                        0.98     8373
   macro avg      0.98    0.98      0.98     8373
weighted avg      0.98    0.99      0.98     8373
```



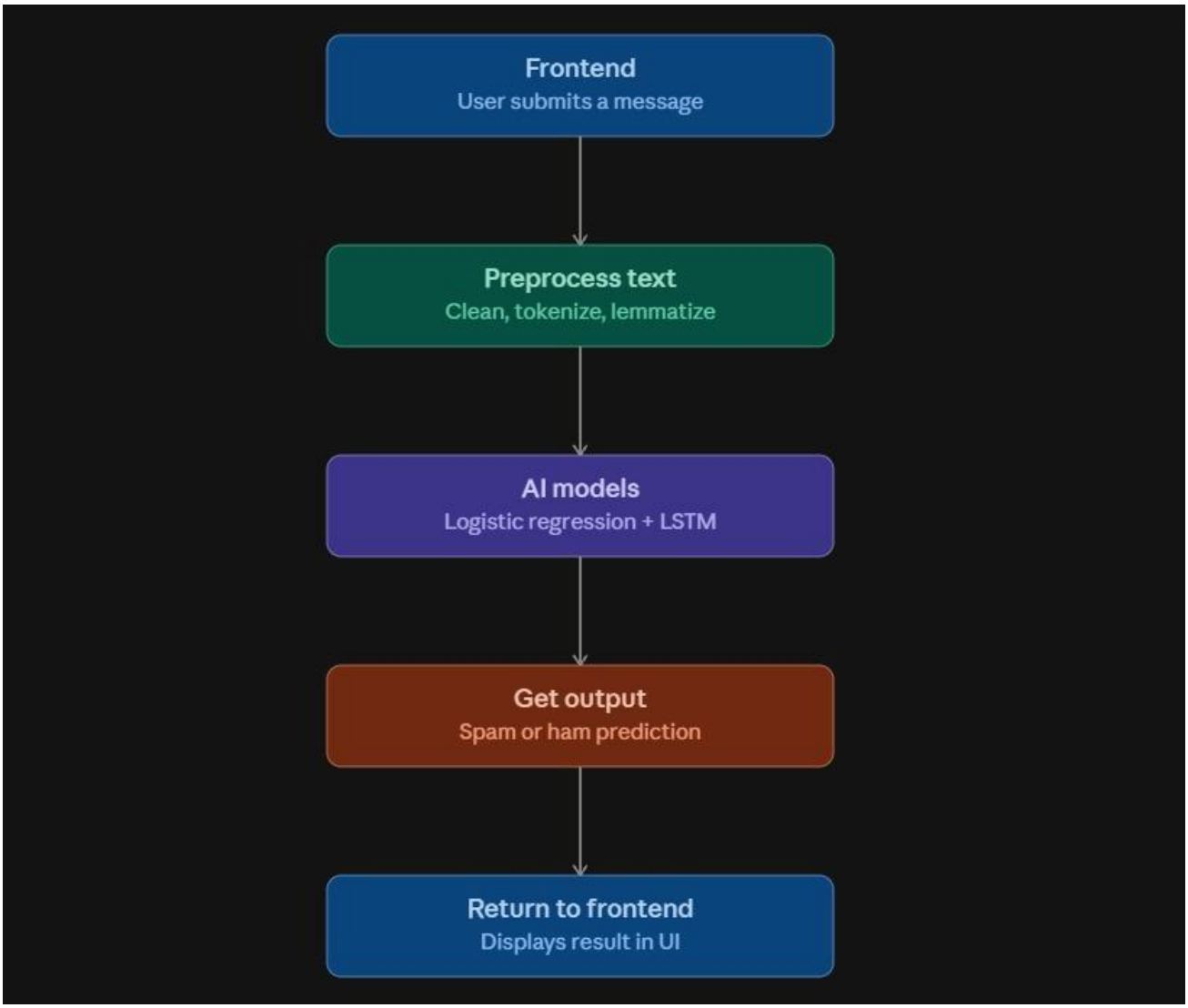


6. Final Application

6.1 System Architecture

The final application is a web-based spam detection system built using Python (Flask/FastAPI backend) and a simple HTML/CSS/JavaScript frontend. The system integrates the best-performing ML and DL models to classify user-provided email or SMS content in real time.

Architecture flow: User Input (Web Browser) → Frontend (HTML/JS) → Backend API (Flask/FastAPI) → NLP Preprocessing Pipeline → ML/DL Model → Classification Result → Frontend Display



6.2 Features

- Text input field for entering or pasting email/SMS content
- Model selection option (ML model or DL model)
- Real-time classification result: Spam or Ham
- Confidence score display
- Clean and responsive user interface

6.3 Application Screenshots

Spam Classifier

Logistic Regression vs LSTM

Message

URGENT: Your account has been suspended. Verify your identity immediately at secure-bank-verify.com or lose access permanently.

Classify

LOGISTIC REGRESSION

Spam

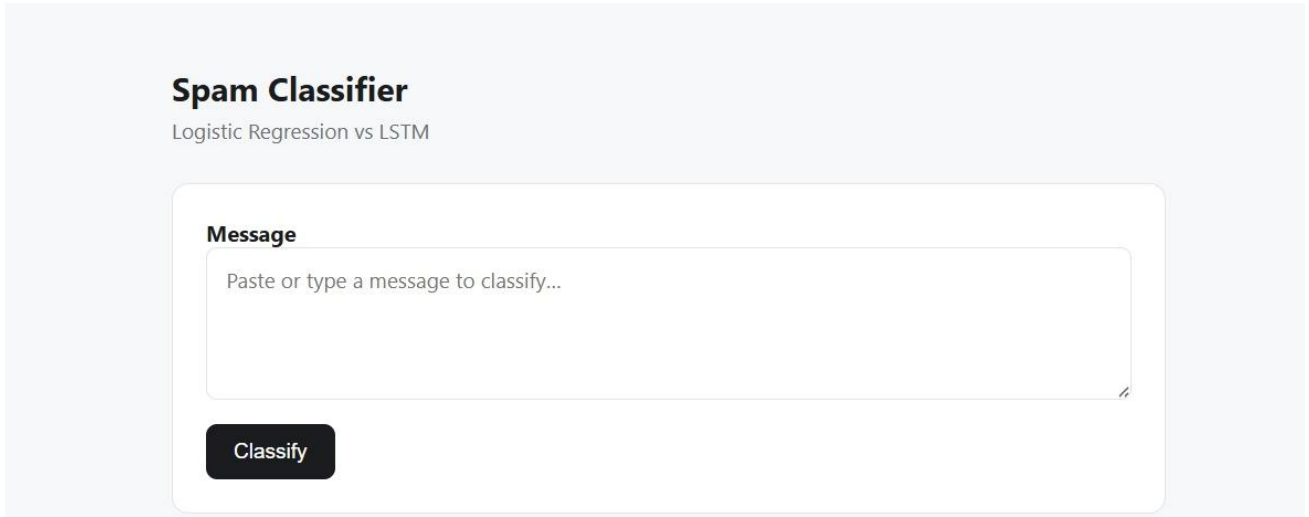
Confidence: 97.55%

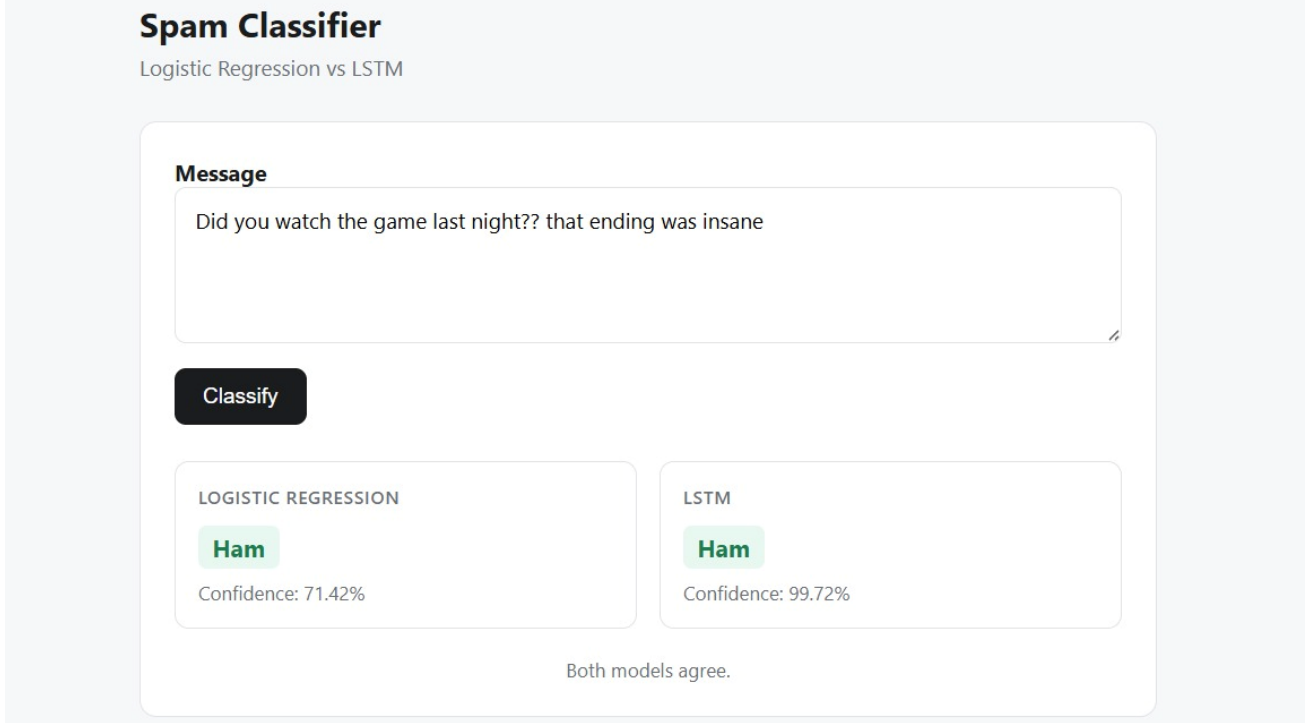
LSTM

Spam

Confidence: 99.85%

Both models agree.


Spam Classifier
Logistic Regression vs LSTM
Message
Paste or type a message to classify...
Classify


Spam Classifier
Logistic Regression vs LSTM
Message
Did you watch the game last night?? that ending was insane
Classify
LOGISTIC REGRESSION
Ham
Confidence: 71.42%
LSTM
Ham
Confidence: 99.72%
Both models agree.

6.4 Workflow

1. User enters or pastes email/SMS text into the input field.
2. The text is submitted to the backend API.
3. The backend preprocesses the text (cleaning, tokenization, vectorization).
4. The preprocessed text is passed to the selected model.
5. The model returns a classification result (Spam/Ham) and confidence score.
6. The result is displayed to the user on the web interface.

7. Ethics and Bias Discussion

7.1 Potential Biases

The training dataset may not accurately represent all types of spam and legitimate messages, which could introduce the following biases:

- Dataset bias: If the training data consists mostly of promotional spam, the model may fail to detect newer spam forms such as phishing or scam emails.
- Language bias: The system is designed for English-language content only; non-English spam would not be detected correctly.
- Temporal bias: Spam patterns evolve over time; a model trained on older data may not detect current spam techniques.
- Class imbalance bias: Ham messages significantly outnumber spam in most datasets, potentially biasing the model toward predicting Ham.

7.2 Ethical Risks

False positives (legitimate emails classified as spam): This could result in users missing important communications such as bank notifications, medical appointments, or university announcements.

False negatives (spam classified as legitimate): This exposes users to phishing links, malware, or fraudulent websites, posing security risks.

Privacy concerns: Email or SMS content may include sensitive personal or financial information. Any data used during development must be handled responsibly and only for educational purposes.

7.3 Social Implications

Automated spam filtering systems that produce false positives could undermine trust in digital communication. Organizations relying on email communications could face operational disruptions if important messages are incorrectly filtered. Additionally, over-reliance on automated systems may reduce human vigilance in identifying suspicious content.

7.4 Mitigation Strategies

- Training on diverse datasets containing a wide variety of spam and ham messages.
- Using multiple evaluation metrics (F1-Score, Precision, Recall) to ensure balanced performance.
- Performing cross-validation and comparing multiple models before deployment.
- Informing users that the system is a decision-support tool; important messages should be reviewed before permanent deletion.
- Handling personal data responsibly in line with data protection principles.
- Regularly retraining the model with updated data to handle evolving spam patterns.

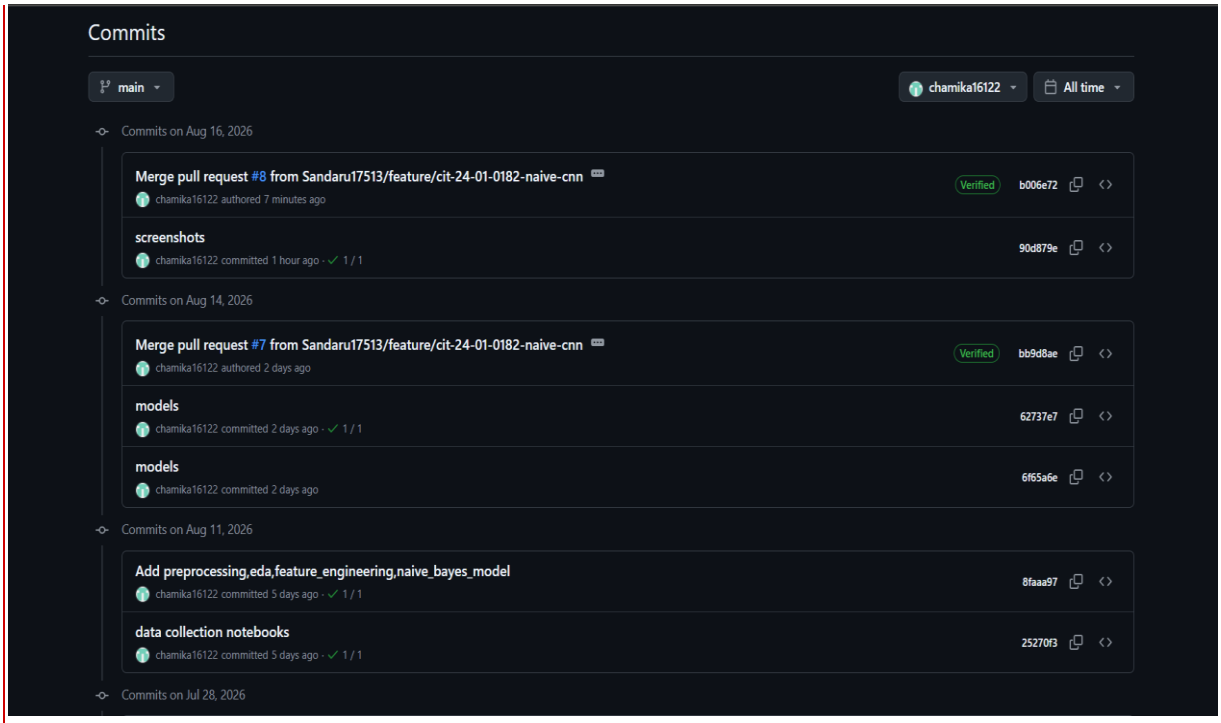
8. Git Contribution Analysis

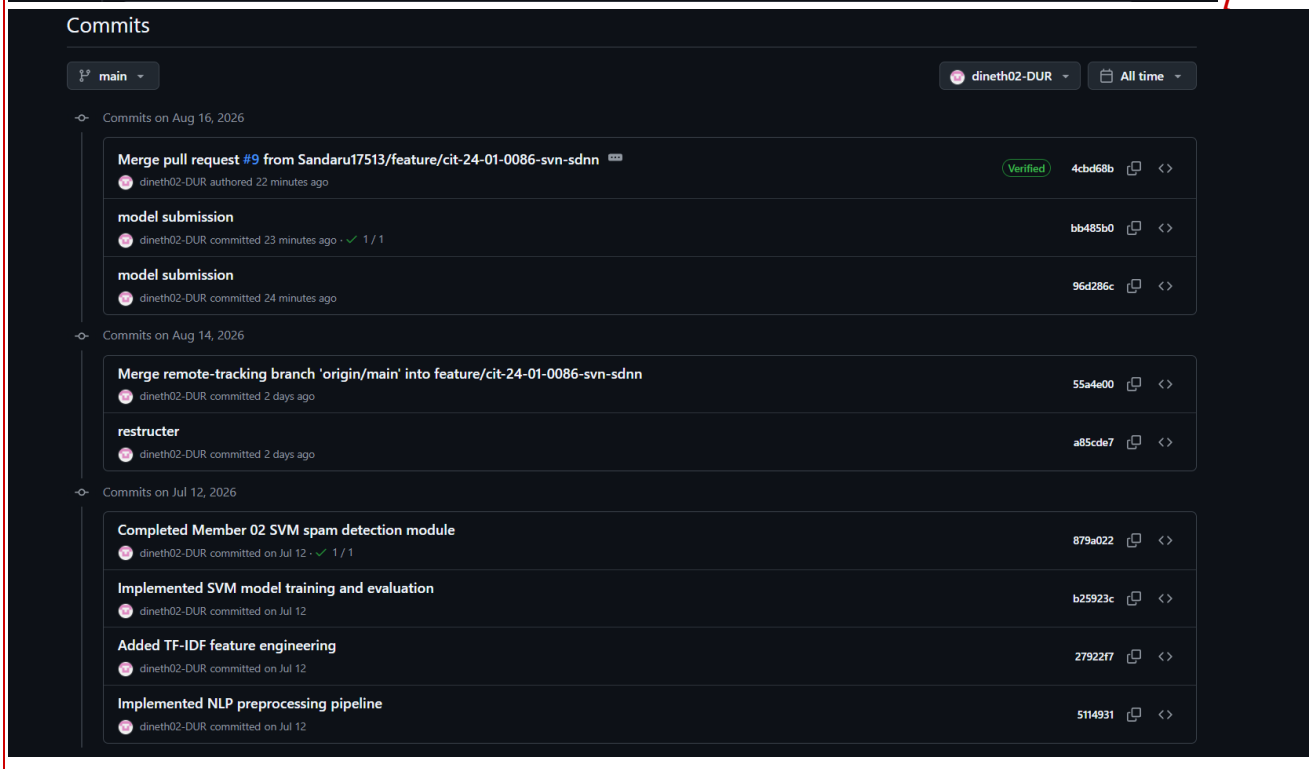
8.1 Repository Overview

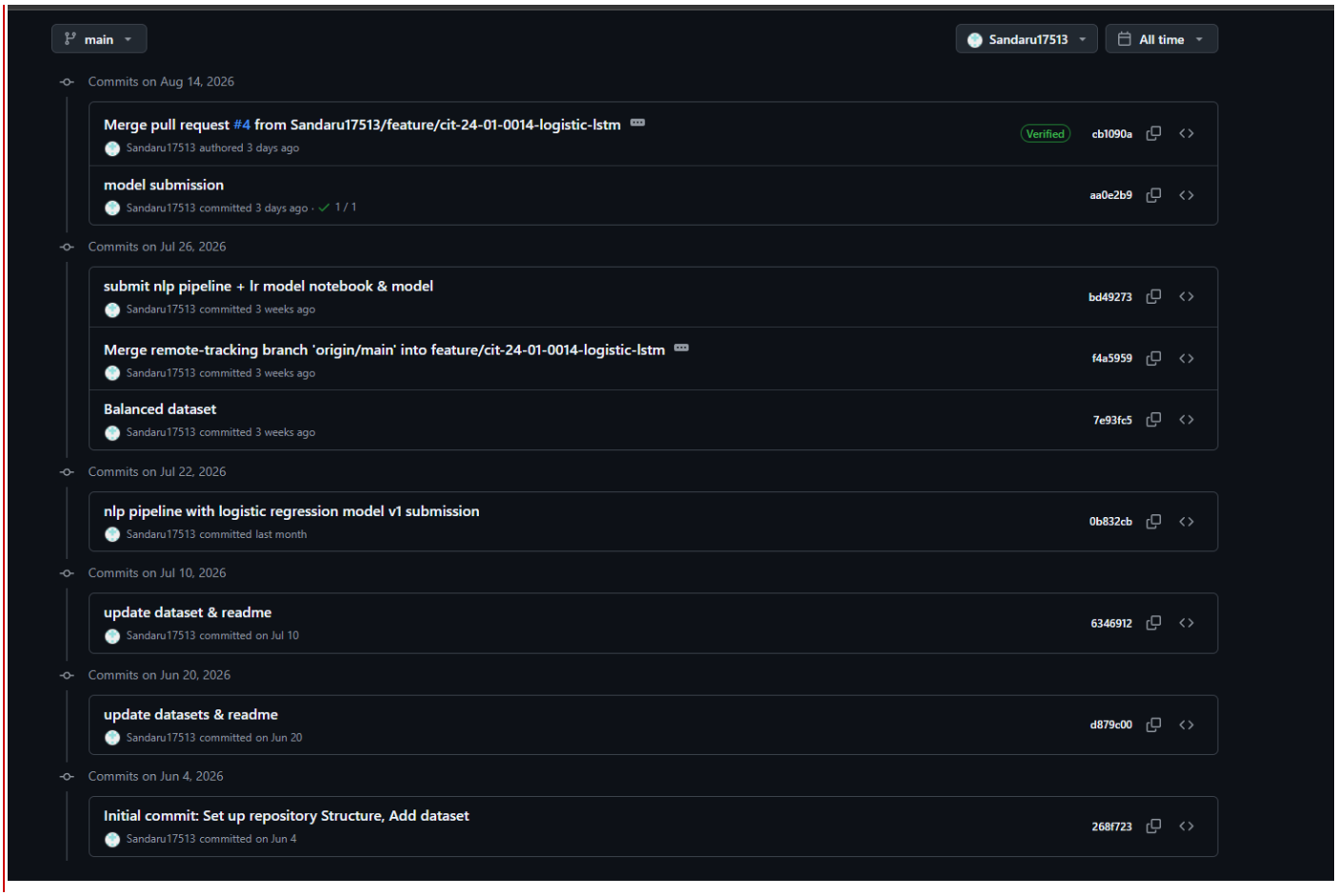
Repository Link: https://github.com/Sandaru17513/NLP_Ctrl-Alt-Elite

Member	Student ID	Branch Name
P. Chamika Janith Piyasena	CIT-24-01-0182	feature/cit-24-01-0182-naive-cnn
B.H. Dineth Ushira Rasanja	CIT-24-01-0086	feature/cit-24-01-0086-svn-sdnn
H. Sandaru Bhagya Senarath	CIT-24-01-0014	feature/cit-24-01-0014-logistic-lstm

8.2 Commit History Screenshots







8.3 Contribution Summary

Task	Member 1	Member 2	Member 3
Dataset Identification	✓		✓
NLP Pipeline Implementation	✓	✓	✓
ML Model (Naïve Bayes / SVM / Logistic Regression)	✓	✓	✓
DL Model (CNN / SDNN / LSTM)	✓	✓	✓
Application Development	✓	✓	
Model Evaluation		✓	
Report Writing			✓
Presentation Preparation	✓	✓	✓
Project Coordination / Leadership	✓		

9. Challenges and Lessons Learned

9.1 Challenges

- Handling class imbalance in the training dataset required oversampling or weighting techniques to avoid biased model predictions.
- Preprocessing diverse data formats (email headers, SMS shorthand, URLs) required extensive cleaning pipelines.
- Training deep learning models (CNN, LSTM, SDNN) required significant computational resources; Google Colab GPU was used to manage training times.
- Integrating multiple individual model notebooks into a single cohesive web application required careful API design.

9.2 Lessons Learned

- Proper preprocessing has a significant impact on model performance — clean data leads to better and more reliable predictions.
- Ensemble approaches and model comparison are essential before deploying any NLP system.
- Git branching and structured collaboration improved team workflow and reduced conflicts.
- Deep learning models require careful hyperparameter tuning and may not always outperform simpler ML models on smaller datasets.

10. Conclusion

This project successfully developed and evaluated a Spam Email/SMS Detection System using Natural Language Processing techniques. Six models were implemented across three group members: Naïve Bayes and CNN (Member 1), SVM and SDNN (Member 2), and Logistic Regression and LSTM (Member 3). A comprehensive NLP pipeline was applied including language detection, text cleaning, tokenization, stop word removal, lemmatization, and vectorization.

The final web application integrates the best-performing model(s) to provide users with real-time spam classification. The system demonstrates that NLP and machine learning techniques can effectively filter spam emails and SMS messages with high accuracy. Future improvements could include retraining with updated datasets to detect emerging spam patterns and expanding the system to support multilingual content.

11. References

[1] Y. Ashfak, "Spam Email Classification Dataset," Kaggle. Available: https://www.kaggle.com/datasets/ashfakyeafi/spam-email-classification/data

[2] Seven Phishing Email Datasets, Figshare. Available: https://figshare.com/articles/dataset/Seven_Phishing_Email_Datasets/25432108

[3] J. Jackksoncsie, "Spam Email Dataset," Kaggle. Available: https://www.kaggle.com/datasets/jackksoncsie/spam-email-dataset

[4] M. Wiechmann, "Enron Spam Data," GitHub. Available: https://github.com/MWiechmann/enron_spam_data

[5] T. Kumar, "SMS Spam Dataset," Kaggle. Available: https://www.kaggle.com/datasets/tinu10kumar/sms-spam-dataset

[6] TheDevastator, "SMS Spam Collection," Kaggle. Available: https://www.kaggle.com/datasets/thedevastator/sms-spam-collection-a-more-diverse-dataset

[7] T. M. Mitchell, Machine Learning. McGraw-Hill, 1997.

[8] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436-444, 2015.

[9] C. C. Aggarwal and C. Zhai, Mining Text Data. Springer, 2012.